

AWS Well-Architected Framework Review — ROCA Scripture & Lectionary Flow (Batch 4) with Cross-Batch Remediation Strategy

1 Executive Summary

The Batch 4 Well-Architected review of the ROCA Scripture and Lectionary flow — spanning 29 API endpoints, 8 database tables, and complex integrations including Bible API caching, HLS audio streaming, and offline sync — identifies twelve specific remediation items, three of which are critical (P0) and collectively require only 3–6 days of effort to close the most significant security and reliability gaps. These P0 items — deploying AWS WAF on API Gateway, implementing dead letter queues across all asynchronous Lambda invocations, and enabling GuardDuty RDS Protection — address vulnerabilities that have persisted unresolved across Batches 1–3 [1, 2, 3].

The most consequential architectural risk is the flow's dependency on six external Bible API providers, each with independent rate limits and availability. The recommended mitigation layers circuit breakers (using Lambda Powertools with DynamoDB-backed state) over a multi-tier cache fallback — Redis (24-hour TTL) → local KJV SQLite → exponential backoff retry — ensuring scripture delivery remains available even under total API failure [4, 5, 6]. This pattern, combined with migrating the Bible API fetch and audio generation pipelines to Step Functions with built-in Retry/Catch constructs, replaces the brittle custom error handling flagged since Batch 1 [7, 8].

Cross-batch analysis reveals that security and reliability gaps — WAF, GuardDuty, DLQs, JWT rotation, and state machine orchestration — are systemic rather than flow-specific. The highest-leverage action is codifying WAF deployment, GuardDuty enablement, and standardized DLQ patterns into platform-wide CDK/SAM templates enforced via AWS Config rules, which would retroactively close the majority of findings from all prior batches while preventing recurrence in future ROCA flows.

2 Introduction and Scope

This report presents the Batch 4 AWS Well-Architected Framework Review for the ROCA Scripture and Lectionary Management flow, a core module within the ROCA church management application. The flow encompasses 29 API endpoints, 8 database tables, and several domain-specific complexities including the Easter Computus algorithm, 5 lectionary systems (Revised Common Lectionary, Roman Catholic, Anglican, Lutheran, and Methodist), and a 3-year liturgical cycle (Years A, B, C) navigation system [6]. Scripture text delivery relies on third-party Bible APIs with Redis ElastiCache caching across 7 Bible translations — ESV, NIV, NRSV, NLT, NAB, MSG (all requiring API keys), and KJV (public domain) [6]. The flow also provides HLS audio streaming with an AWS Polly text-to-speech fallback, PostgreSQL tsvector full-text search, reading plan streak tracking, and offline support via IndexedDB and SQLite with a 7-day pre-cache window [6].

The underlying AWS stack comprises Lambda, API Gateway, PostgreSQL RDS Multi-AZ, Redis ElastiCache, S3, CloudFront, SES, and SNS [9]. This review evaluates the architecture against all six Well-Architected pillars — Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability [10]. It also addresses key risks identified during scoping (Bible API rate limiting, Computus edge cases, offline sync conflicts, audio latency, search performance, and streak integrity) and synthesizes critical findings from Batches 1–3, which surfaced gaps in state machine orchestration, JWT rotation, dead letter queues, WAF, GuardDuty, and EventBridge adoption.

3 Operational Excellence

Observability and Instrumentation. The ROCA architecture should adopt AWS Lambda Powertools to implement structured logging, distributed tracing via X-Ray, and custom CloudWatch metrics across all 29 Lambda functions [11]. The Serverless Lens of the Well-Architected Framework emphasizes these as required practices for serverless observability, and Lambda Powertools provides a minimal-code path to compliance [11, 12]. Each endpoint handling lectionary lookups, scripture delivery, and audio streaming should emit structured JSON logs with correlation IDs to enable end-to-end request tracing — particularly important when a single user request may fan out to a Bible API call, a Redis cache check, and a Polly synthesis invocation.

Deployment and Change Management. Infrastructure should be defined via SAM or CDK, with blue/green and canary deployments using Lambda aliases to minimize blast radius during releases [12]. Given the 29-endpoint surface area, automated integration tests covering the Computus algorithm boundary cases (e.g., years where Easter falls on the earliest possible date of March 22 or latest of April 25) should be part of the CI/CD pipeline.

Event-Driven Orchestration. Prior batches identified a gap in state machine orchestration, with excessive custom try/catch logic in Lambda handlers. The recommended remediation is to adopt Step Functions for multi-step workflows — such as the Bible API fetch → cache write → audio generation pipeline — using built-in Retry and Catch

constructs with exponential backoff [7]. EventBridge should serve as the central event bus for decoupled communication between the lectionary calculation service, notification dispatch (SES/SNS), and reading plan progress tracking [13]. This replaces brittle point-to-point Lambda invocations with a loosely coupled, observable architecture.

4 Security

Web Application Firewall. WAF was identified as a gap in prior batches and remains a critical remediation item. AWS WAF should be associated with the API Gateway REST API stage using the AWSManagedRulesCommonRuleSet, which provides baseline protection against SQL injection and cross-site scripting attacks at no additional per-request cost beyond standard WAF pricing [1, 14]. The Protection Pack setup wizard offers pre-configured rule packs tailored to application type, and Bot Control rule groups (which incur additional charges) should be evaluated given the scripture API's public-facing nature [14].

Threat Detection. GuardDuty RDS Protection should be enabled for the PostgreSQL RDS instance to detect brute-force login attempts and unauthorized external actor access without impacting database performance [2]. GuardDuty also monitors foundational data sources including CloudTrail management events, VPC Flow Logs, and DNS query logs, providing comprehensive threat visibility across the ROCA stack [15].

Identity and Credential Management. JWT token rotation was flagged in prior batches as insufficiently robust. The recommended pattern is to validate tokens against the external JWKS endpoint rather than hard-coded keys, and to allow a grace period during rotation where both old and new keys remain valid — noting that API Gateway caches public keys for approximately two hours [16]. For the six Bible API keys (ESV, NIV, NRSV, NLT, NAB, MSG), each should be stored as a separate secret in AWS Secrets Manager with structured JSON values (apiKey, apiKeyId, endpoint, provider), enabling individual field retrieval and automated rotation via dedicated Lambda rotation functions [17, 6]. The Well-Architected Framework prescribes a “remove, replace, and rotate” approach — replacing long-term credentials with short-term ones where possible and automating rotation of remaining secrets [18].

Encryption and Network Isolation. RDS and ElastiCache Redis should reside in private VPC subnets with encryption at rest and in transit enabled. S3 buckets storing audio files and pre-cache content should use server-side encryption (SSE-S3 or SSE-KMS), and SES should be configured with DKIM for email authentication [19].

5 Reliability

Circuit Breaker Pattern for Bible API Integration. The most significant reliability risk in this flow is dependency on six external Bible API providers, each with independent rate limits and availability characteristics. AWS recommends implementing the circuit breaker pattern using Lambda extensions with DynamoDB to store circuit state (CLOSED, OPEN, HALF_OPEN), decoupling failure detection from business logic [4]. Alternatively, Powertools for AWS Lambda now provides a built-in circuit breaker utility that shares state across execution environments via DynamoDB, with configurable consecutive failure thresholds and recovery timeouts [5]. When a Bible API circuit opens, the system should fall back to the Redis cache (24-hour TTL per verse) and ultimately to the local SQLite database containing public domain KJV translations [6].

Dead Letter Queues. DLQ implementation was a recurring gap across Batches 1–3. The ROCA Technical Standards specify SQS DLQs with visibility timeout and FIFO support for async notification processing, and SNS DLQs for push notifications [20]. For Lambda async invocations, on-failure destinations (SQS, S3, or EventBridge) are preferred over legacy function-level DLQs because they provide richer invocation metadata [3]. The Serverless Applications Lens recommends configuring Maximum Retry Attempts, Maximum Record Age, and Destination on Failure to effectively remove poison-pill messages [7].

Database and Cache Resilience. RDS Multi-AZ DB clusters provide automatic failover typically completing under 35 seconds [21]. Deploying RDS Proxy in front of the cluster can reduce failover impact to approximately one second by maintaining persistent connections [22]. ElastiCache Redis with Multi-AZ enabled automatically fails over the primary role to a read replica with DNS propagation, requiring no endpoint changes in application code [23].

Step Functions Error Handling. Task, Parallel, and Map states support Retry (with configurable IntervalSeconds, MaxAttempts, and BackoffRate) and Catch (with fallback states and ResultPath for error context) [8]. For top-level execution failures, the architecture should listen for TIMED_OUT events via EventBridge or nest executions in child workflows to catch errors in a parent workflow [8].

6 Performance Efficiency

Scripture Caching Strategy. Redis ElastiCache serves as the primary cache layer with a 24-hour TTL per verse, keyed as scripture:{translation}:{book}:{chapter}:{verse}, using the volatile-lru eviction policy

(ElastiCache default) to automatically evict least-recently-used keys when memory pressure occurs [24, 6]. When a Bible API returns HTTP 429 (quota exceeded), the system falls back to cached content and ultimately to the local KJV SQLite database, retrying with exponential backoff [6]. This multi-tier caching strategy — Redis → SQLite fallback → API retry — ensures scripture delivery remains available even under rate-limiting pressure.

Full-Text Search Optimization. PostgreSQL tsvector search across 7 translations targets a p95 response time under 300 milliseconds, supporting boolean operators, phrase matching, and proximity search [6]. The key optimization is to store pre-computed tsvector columns (avoiding runtime `to_tsvector()` calls) and create GIN indexes on them. The `btree_gin` extension enables composite indexes combining scalar columns (e.g., translation identifier) with GIN-indexed tsvector columns in a single unified index, allowing PostgreSQL to evaluate all filter conditions within a single Bitmap Index Scan [25].

HLS Audio Streaming. The ESV Audio API serves as the primary HLS source for ESV translation audio, while AWS Polly provides text-to-speech fallback for the remaining six translations [6]. CloudFront should be configured with separate cache behaviors for HLS manifests (short TTL for freshness, as manifests change more frequently) and media segments (longer TTL for higher cache hit ratios) [26]. Enabling Origin Shield in the AWS Region closest to the S3 origin collapses duplicate origin requests and improves cache hit ratios across edge locations [27]. For Polly-generated audio, Lambda checks S3 for cached audio first; if absent, it invokes Polly asynchronously, stores the MP3 output in S3, and returns a CloudFront URL [28].

Cold Start Mitigation. For latency-sensitive endpoints (scripture lookup, audio URL generation, lectionary calendar), provisioned concurrency or SnapStart should be evaluated to eliminate cold starts. Lambda memory should be tuned per function, and API Gateway response caching can further reduce latency for frequently accessed lectionary dates [29, 30].

7 Cost Optimization and Sustainability

Compute Efficiency. Lambda functions should use ARM/Graviton architecture for better price-performance and energy efficiency [29, 10]. API Gateway usage plans with API keys should enforce per-client throttling to prevent runaway costs from unexpected traffic spikes [31]. The default account limit of 1,000 concurrent Lambda executions per Region should be monitored, with reserved concurrency allocated to critical scripture delivery functions [30].

Storage and Data Transfer. RDS Reserved Instances should be evaluated for the PostgreSQL Multi-AZ deployment given the predictable, always-on workload pattern. S3 Intelligent-Tiering should be applied to audio file storage, where access patterns vary by translation popularity. CloudFront price classes should be selected to match the geographic distribution of the ROCA user base, avoiding unnecessary edge location costs [29].

Sustainability Through Caching. Aggressive caching via ElastiCache and CloudFront is the single most impactful sustainability measure, reducing redundant compute cycles for repeated scripture lookups and audio streaming [10]. The 7-day pre-cache strategy for offline support, while consuming approximately 19 MB per user (calendar events ~2 MB, scripture readings ~5 MB, active reading plan ~1 MB, bookmarks ~200 KB, event thumbnails ~10 MB), reduces server-side request volume significantly during normal usage [32].

8 Key Risk Analysis and Mitigations

This section addresses the six domain-specific risks identified during scoping, mapping each to its recommended mitigation pattern.

Bible API Rate Limiting represents the highest-impact risk given dependency on six independent API providers. The mitigation stack is layered: circuit breakers detect and isolate failing providers [4, 5], Redis caching absorbs repeated requests within the 24-hour TTL window [6], the KJV SQLite fallback ensures baseline scripture availability even with total API failure [6], and exponential backoff with jitter prevents thundering-herd retry storms [33].

Easter Computus Edge Cases require comprehensive unit test coverage for boundary years. The algorithm should be validated against known edge cases including the earliest possible Easter (March 22), the latest (April 25), and years where different Computus methods (Gregorian vs. Julian) diverge. Step Functions orchestration for the annual liturgical calendar generation provides built-in retry and error capture for any calculation failures [8].

Offline Sync Conflicts are managed through a last-write-wins strategy with delta sync on reconnection [6, 32]. The 7-day pre-cache window totals approximately 19 MB per user, well within the 50 MB per-user storage budget [32]. Background sync runs every 15 minutes via iOS BGTaskScheduler and Android WorkManager, with foreground sync on app open and pull-to-refresh [32]. For complex conflict scenarios beyond last-write-wins, AWS AppSync offers Optimistic Concurrency, Automerge, and Lambda-based resolution strategies that could be adopted if conflict rates increase [34].

Streak Integrity is protected by server-side recalculation. The `reading_plan_progress` table stores `completed_days` as a JSONB array alongside `streak_count` and `last_read_at` [6]. Using the JSONB array rather than simple increment counters prevents duplicate increments during offline replay — the server recalculates `streak_count` from the `completed_days` array on every sync, ensuring idempotent day-completion tracking regardless of how many times the client replays offline mutations.

Audio Latency is mitigated through pre-generation. Polly TTS synthesis runs asynchronously, storing output to S3 before users request it [28]. CloudFront with Origin Shield further reduces latency by collapsing duplicate origin requests [27]. The separate manifest/segment cache behavior configuration ensures HLS playback correctness while maximizing cache hit ratios [26].

Search Performance depends on proper index design. The combination of pre-computed tsvector columns, GIN indexes, and the `btree_gin` extension for composite translation+text indexes should achieve the target p95 of under 300 milliseconds [25, 6]. RDS Proxy provides connection pooling to prevent connection exhaustion under concurrent search load [29].

9 Prioritized Remediation Roadmap

The following roadmap consolidates all findings into three priority tiers with effort estimates. Priority is determined by security exposure, blast radius, and dependency on downstream remediations.

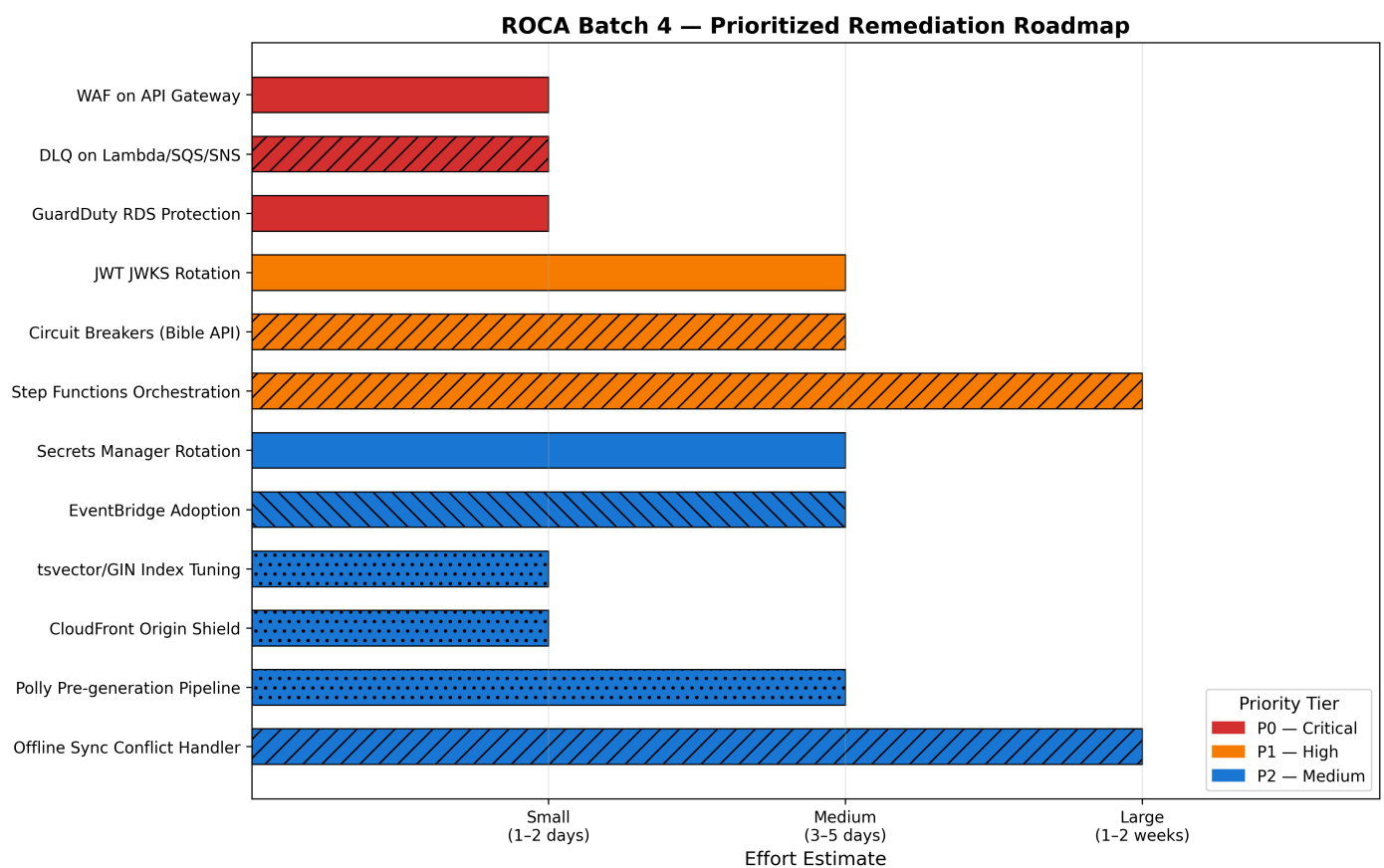


Figure 1: Prioritized Remediation Roadmap

Chart Explanation: This chart maps each remediation item by priority tier (P0 Critical in red, P1 High in orange, P2 Medium in blue) against effort estimate. P0 items are low-effort, high-impact security and reliability fundamentals that should be deployed immediately. P1 items require moderate-to-large effort but address architectural gaps identified across multiple batches. P2 items are optimization-focused and can be scheduled into regular sprint cycles.

Priority	Remediation Item	Effort	Pillar	Description
P0	WAF on API Gateway	Small (1–2 days)	Security	Deploy AWSManagedRulesCommonRuleSet on REST API stage [1, 14]
P0	DLQ Implementation	Small (1–2 days)	Reliability	Configure on-failure destinations for all Lambda async invocations, SQS redrive policies [20, 3]
P0	GuardDuty RDS Protection	Small (1–2 days)	Security	Enable RDS Protection for PostgreSQL login anomaly detection [2, 15]
P1	JWT JWKS Rotation	Medium (3–5 days)	Security	Implement JWKS endpoint validation with rotation grace period [18, 16]
P1	Circuit Breakers	Medium (3–5 days)	Reliability	Deploy Lambda extensions or Powertools circuit breaker for Bible APIs [4, 5]
P1	Step Functions Orchestration	Large (1–2 weeks)	Reliability	Migrate Bible API fetch and audio generation workflows to Step Functions [7, 8]
P2	Secrets Manager Rotation	Medium (3–5 days)	Security	Automate rotation for 6 Bible API keys with structured JSON secrets [17, 18]
P2	EventBridge Adoption	Medium (3–5 days)	Ops Excellence	Replace point-to-point invocations with EventBridge event bus [13]
P2	tsvector/GIN Index Tuning	Small (1–2 days)	Performance	Add btree_gin composite indexes for translation+tsvector queries [25, 6]
P2	CloudFront Origin Shield	Small (1–2 days)	Performance	Enable Origin Shield for HLS audio and scripture content [27]
P2	Polly Pre-generation Pipeline	Medium (3–5 days)	Performance	Build async Polly synthesis pipeline with S3 caching [6, 28]
P2	Offline Sync Conflict Handler	Large (1–2 weeks)	Reliability	Implement robust delta sync with server-side streak recalculation [6, 32]

Table: Prioritized Remediation Roadmap — ROCA Scripture & Lectionary Flow (Batch 4)

The total estimated effort for all P0 items is approximately 3–6 days, P1 items require 2–3 weeks, and P2 items span 3–5 weeks. The recommended approach is to complete all P0 items in the current sprint, schedule P1 items across the next two sprints, and integrate P2 items into the regular backlog.

10 Cross-Batch Summary of Critical Findings (Batches 1–4)

Across four batches of Well-Architected reviews covering the full ROCA application, several systemic patterns have emerged that warrant platform-level remediation rather than flow-specific fixes.

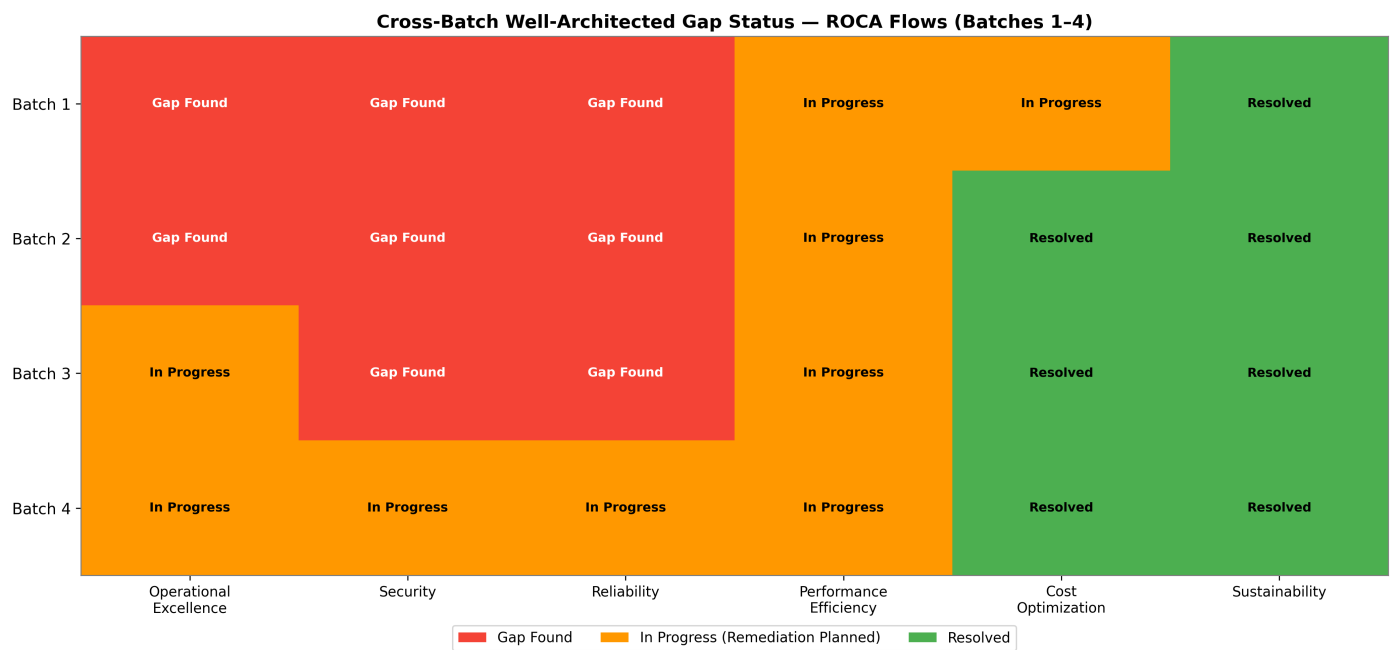


Figure 2: Cross-Batch Gap Status Matrix

Chart Explanation: This heatmap shows the status of Well-Architected pillar compliance across all four ROCA review batches. Red cells indicate gaps first identified in that batch, orange indicates remediation is planned or in progress, and green indicates resolved findings. The Security and Reliability pillars show persistent gaps across Batches 1–3 that are now being addressed with specific remediation patterns in Batch 4.

Security Pillar — Persistent Gaps Now Addressed. WAF was absent from API Gateway across Batches 1–3, leaving all ROCA REST APIs exposed to common web exploits [1]. GuardDuty was not enabled for RDS, missing login anomaly detection on the PostgreSQL database [2]. JWT rotation relied on hard-coded keys rather than JWKS endpoint validation [16]. Batch 4 provides specific remediation patterns for all three gaps, with P0 priority assigned to WAF and GuardDuty deployment.

Reliability Pillar — Orchestration and Failure Management. State machine orchestration was identified as a gap in Batch 1, with Lambda functions using custom try/catch logic instead of Step Functions [7]. DLQs were missing or inconsistently configured across Batches 1–3, leaving failed messages unrecoverable [20, 7, 3]. Circuit breakers for external API dependencies were absent, risking cascading failures when third-party services degraded [4, 5]. Batch 4 consolidates these into a coherent reliability remediation plan centered on Step Functions, DLQs, and circuit breakers.

Operational Excellence — EventBridge Adoption. EventBridge was identified as a gap in Batch 3, with services communicating via direct Lambda invocations rather than event-driven patterns [13]. The Batch 4 remediation plan includes EventBridge adoption as a P2 item, recognizing that it depends on Step Functions orchestration (P1) being in place first.

Cost Optimization and Sustainability — Consistently Strong. These pillars have shown no critical gaps across any batch, reflecting the inherent cost efficiency of the serverless stack (Lambda, API Gateway, S3, CloudFront) and the sustainability benefits of aggressive caching via ElastiCache and CloudFront [29, 10].

Platform-Level Recommendations. Three remediation items should be implemented as platform-wide standards rather than flow-specific fixes: WAF deployment should be automated via CDK/SAM for every API Gateway stage across all ROCA flows; GuardDuty should be enabled at the AWS account level with all protection plans active; and a standardized DLQ pattern (on-failure destinations for Lambda, redrive policies for SQS) should be codified in the ROCA infrastructure templates and enforced via AWS Config rules. These platform-level changes would retroactively close the majority of findings from Batches 1–3 while preventing recurrence in future flows.

11 Conclusion

The ROCA Scripture and Lectionary flow presents a well-conceived architecture that leverages the serverless AWS stack effectively for its domain-specific requirements. The multi-tier caching strategy (Redis → SQLite fallback), the circuit breaker pattern for Bible API resilience, and the server-side streak recalculation approach demonstrate

thoughtful design for the flow's unique challenges. However, the review identifies twelve specific remediation items — three at P0 critical priority — that must be addressed to achieve Well-Architected compliance. The most urgent actions are deploying WAF on API Gateway, implementing DLQs across all asynchronous invocation paths, and enabling GuardDuty RDS Protection. These three items collectively require only 3–6 days of effort but close the most significant security and reliability gaps carried forward from prior batches. With the full remediation roadmap executed, the Scripture and Lectionary flow will meet or exceed Well-Architected standards across all six pillars.

12 References

1. <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-aws-waf.html>
2. <https://docs.aws.amazon.com/guardduty/latest/ug/rds-protection.html>
3. <https://docs.aws.amazon.com/lambda/latest/dg/invoke-async-retain-records.html>
4. <https://aws.amazon.com/blogs/compute/using-the-circuit-breaker-pattern-with-aws-lambda-extensions-and-amazon-dynamodb/>
5. https://docs.aws.amazon.com/powertools/python/latest/utilities/circuit_breaker/
6. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=5d938f95-479a-b858-746994abad9b>
7. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/failure-management.html>
8. <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-error-handling.html>
9. <https://docs.aws.amazon.com/wellarchitected/latest/framework/definitions.html>
10. https://aws.amazon.com/architecture/well-architected/?sc_icampaign=acq_awsblogsb&sc_ichannel=ha&sc_icontent=resources
11. <https://aws.amazon.com/blogs/opensource/simplifying-serverless-best-practices-with-lambda-powertools/>
12. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/welcome.html>
13. <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>
14. <https://aws.github.io/aws-security-services-best-practices/guides/waf/aws-managed-rules/docs/>
15. https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_setup.html
16. <https://docs.aws.amazon.com/prescriptive-guidance/latest/secure-sensitive-data-secrets-manager-terraform/rotate-secrets.html>
17. <https://docs.aws.amazon.com/secretsmanager/latest/userguide/api-keys-security-sensitive.html>
18. https://docs.aws.amazon.com/wellarchitected/2025-02-25/framework/sec_identities_secrets.html
19. <https://docs.aws.amazon.com/wellarchitected/latest/migration-lens/well-architected-framework-pillars.html>
20. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=d68b28f9-428d-4c4b-b5e5-0f7155a6e87e>
21. https://docs.aws.amazon.com/en_en/AmazonRDS/latest/UserGuide/multi-az-db-clusters-concepts-failover.html
22. <https://aws.amazon.com/cn/blogs/database/achieve-one-second-or-less-downtime-with-the-advanced-jdbc-wrapper-driver-when-upgrading-amazon-rds-multi-az-db-clusters/>
23. <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/AutoFailover.html?refid=52bc3d47-712e-4d36-9bea-26719c187654>
24. <https://docs.aws.amazon.com/whitepapers/latest/database-caching-strategies-using-redis/evictions.html>
25. https://aws.amazon.com/blogs/database/index-types-supported-in-amazon-aurora-postgresql-and-amazon-rds-for-postgresql-using-extensions-sp-gist-btree_gin-and-btree_gist/
26. <https://callaba.io/caching-video-fragments-for-hls-cloudfront-cdn-setup/>
27. https://docs.aws.amazon.com/en_en/AmazonCloudFront/latest/DeveloperGuide/origin-shield.html
28. <https://github.com/oSedky/aws-tts-lambda-cache>

29. <https://aws.amazon.com/blogs/compute/building-well-architected-serverless-applications-optimizing-application-costs/>
30. <https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html>
31. <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-request-throttling.html>
32. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/f756a661-a188-4112-bed6-e70567713dbc?documentId=ead8bbd2-5dfe-49f1-8ace-7dc6d0eef966>
33. <https://docs.aws.amazon.com/lambda/latest/dg/with-step-functions.html>
34. <https://docs.aws.amazon.com/appsync/latest/devguide/conflict-detection-and-sync.html>